Three-Stage Bearing Health Classification Using Vibration STFT and Temperature Trends in a Run-to-Failure Setting

Long Truong[1*†] and Phu Le Nguyen[2†]

[1]FPT University, Ho Chi Minh City, Vietnam.
[2]Ho Chi Minh City Open University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): longt5@fpt.edu.vn;
Contributing authors: phunguyen07d1@gmail.com;
†These authors contributed equally to this work.

Abstract

Condition-based maintenance (CBM) for rotating machinery requires not only detecting faults but also recognizing early degradation to enable timely intervention. This paper studies three-stage bearing health classification (Healthy, Degrading, Fault) using a run-to-failure dataset with two-axis vibration and temperature measurements. To better reflect deployment and avoid overly optimistic estimates caused by temporal correlation, we adopt a **leakage-aware evaluation** aligned with the time-to-failure (TTF) axis, where training samples come from earlier life segments and testing samples come from later segments. This protocol explicitly mitigates the well-known leakage from *overlapped windows* when random/window-level splits are used. Our approach converts vibration windows into time–frequency representations using short-time Fourier transform (STFT) and learns discriminative patterns with a lightweight 2D CNN. In parallel, temperature channels are summarized into compact trend/statistical descriptors and fused with vibration features via a late-fusion classifier. We report macro-averaged F1 and per-class precision/recall to account for class imbalance across life stages and compute temporal metrics over *present classes* for late-life slices. In a development (file-wise stratified) setting, the model reaches Macro-F1 $\approx$ **0.7762** with per-class F1 of Healthy $\approx$ **0.69**, Degrading $\approx$ **0.64**, and Fault = **1.00**. In a leakage-aware temporal evaluation focused on late-life behavior where only Degrading/Fault are present, the model achieves Accuracy $\approx$ **0.95** and Macro-F1 $\approx$ **0.9467** over the present classes. Our study is a *proof of concept* on a single run; we highlight the evaluation protocol and multi-modal fusion as

practical choices for realistic deployment and outline validation on additional datasets as future work.

Keywords: bearing health monitoring, run-to-failure, degradation stage classification, STFT spectrogram, vibration–temperature fusion, temporal split, condition-based maintenance

1 Introduction

Rotating machinery is a core asset in modern industry, and rolling-element bearings are among the most failure-prone components due to continuous mechanical stress, friction, and thermal loading. Unexpected bearing failures can cause unplanned downtime, safety risks, and substantial maintenance costs. Consequently, *condition-based maintenance* (CBM) and *prognostics and health management* (PHM) aim to continuously monitor bearing health and trigger maintenance actions before critical breakdowns.

Vibration-based diagnosis has been widely explored, where deep learning models—especially convolutional neural networks (CNNs)—learn discriminative patterns from raw waveforms or time–frequency representations such as spectrograms. However, practical CBM deployment often needs more than binary "healthy vs. fault" decisions. In run-to-failure monitoring, an intermediate *degrading* stage is crucial for early warning, but it is also the hardest to recognize due to subtle signal changes, class imbalance, and evolving operating conditions [1].

A second challenge is *evaluation realism*. Many bearing diagnostic studies rely on random or window-level splits that inadvertently leak temporally adjacent samples across train/test, yielding overly optimistic performance [2, 3]. In real deployments, models must generalize from earlier-life behavior to later-life degradation patterns that were not observed during training. Therefore, leakage-aware temporal protocols aligned with the time-to-failure (TTF) axis are necessary to estimate true field performance.

Finally, multi-sensor information can improve stage recognition. Temperature is often correlated with friction and abnormal thermal behavior, providing complementary evidence to vibration signals, especially near degradation onset [4]. Yet effectively fusing temperature with vibration while keeping the model lightweight remains non-trivial.

In this work, we address three-stage bearing health classification (Healthy, Degrading, Fault) under a run-to-failure setting using vibration STFT representations and temperature trend descriptors [1, 5]. The key idea is to couple (i) time–frequency CNN features from vibration with (ii) compact temperature trend/statistical descriptors via a late-fusion classifier, and (iii) validate performance using leakage-aware temporal splits aligned with TTF. To bridge development and deployment, we follow a two-phase workflow: we first train a baseline model under a file-wise stratified split, and then perform leakage-aware temporal fine-tuning aligned with the TTF axis (train [0,60]%, val [60,70]%, test [70,100.1]%). This design explicitly tests generalization from earlier-life behavior to later-life degradation while reducing the optimistic bias caused by temporally overlapped windows. **Contributions.** This paper makes the following contributions:

- We formulate a three-stage bearing health classification problem on a run-to-failure dataset with synchronized two-axis vibration and temperature measurements [1, 5].
- This two-phase scheme (stratified pre-training → temporal fine-tuning) provides a clearer separation between development validation and deployment-oriented testing, preventing temporal leakage across overlapping windows.
- We adopt and emphasize a leakage-aware evaluation protocol aligned with the TTF timeline, which better matches deployment and mitigates optimistic bias from temporal leakage [2, 3].

2 Related Work

2.1 Bearing fault diagnosis with deep learning and time–frequency representations

Deep learning has become a dominant approach for rolling-element bearing diagnosis due to its ability to learn discriminative patterns directly from sensor signals. A common pipeline transforms vibration windows into 2D time–frequency images (e.g., STFT spectrograms or wavelet scalograms) and applies 2D CNNs originally developed for vision tasks. Verstraete et al. [6] demonstrated that time–frequency image analysis combined with CNNs can achieve strong fault diagnosis performance and influenced subsequent spectrogram/scalogram-based methods. More recent studies continue to explore CNN architectures and feature learning strategies for robust vibration-based fault recognition [7].

While these methods are effective for identifying explicit fault signatures, run-to-failure monitoring often requires recognizing an intermediate degradation stage in addition to the final fault stage. This setting introduces higher ambiguity and stronger class imbalance, making Degrading detection more challenging than binary fault classification.

We adapt parts of an open-source CNN implementation for bearing diagnosis [8], and extend it with temperature fusion, STFT normalization tailored to run-to-failure windows, and leakage-aware TTF-aligned evaluation.

2.2 Run-to-failure data and degradation-stage recognition

Run-to-failure datasets provide realistic progression of health states but also raise challenges in label definition and evaluation. The bearing run-to-failure dataset used in this work includes vibration and temperature streams over the asset lifetime [1, 5]. Prior studies on run-to-failure monitoring frequently rely on stage labels derived from time or degradation indicators, aiming to approximate early-life, mid-life, and late-life behaviors. However, the intermediate stage is often under-represented and may overlap with healthy behavior, which motivates integrating complementary sensing modalities and carefully designing evaluation protocols.

Stage-based monitoring vs. RUL prognostics.

Beyond fault recognition, PHM research often targets remaining useful life (RUL) estimation using regression or sequence models. However, reliable RUL supervision and

consistent degradation indicators are not always available in practical run-to-failure experiments. As an alternative, stage-based health classification provides actionable early-warning signals with weaker supervision, which aligns with many CBM decision processes. In this work, we focus on three-stage health recognition to support timely maintenance actions under realistic labeling constraints.

2.3 Evaluation protocols and data leakage in bearing diagnostics

A growing body of work highlights that random splitting of highly overlapped windows can cause severe information leakage: temporally adjacent windows share similar spectral content, and if they appear in both training and testing, performance can be inflated [2]. Vieira et al. [3] further argue for more realistic evaluation strategies in machine diagnostics that reflect the operational reality of future data being unseen at training time. These findings motivate leakage-aware temporal splits aligned with the time-to-failure axis, which we adopt as a primary evaluation protocol to better approximate deployment performance. In addition to temporal leakage, generalization across different bearings or operating conditions (domain shift) remains challenging in machine diagnostics. While fully bearing-wise evaluation may require larger cross-unit coverage, our TTF-aligned temporal protocol is a necessary first step toward deployment realism by preventing near-duplicate windows from leaking across splits.

2.4 Multi-sensor fusion with temperature and vibration

Multi-sensor learning is a practical direction for improving robustness, as temperature may capture thermal effects associated with friction, lubrication issues, or abnormal contact conditions. Huang et al. [4] showed that combining temperature-related signals with vibration features can improve diagnostic performance compared with single-sensor baselines. Inspired by this direction, we fuse vibration time–frequency CNN features with compact temperature trend/statistical descriptors, aiming to improve Degrading-stage recognition in run-to-failure monitoring while keeping the model lightweight.

3 Dataset and Problem Definition

3.1 Run-to-failure dataset

We use the run-to-failure bearing dataset provided by Jung et al. [1, 5]. The dataset contains **one run** (`RUNS=1`) consisting of **129 files** (`FILES=129`). Two-axis vibration signals are sampled at **25.6 kHz**.

3.2 Segmentation and feature construction

We segment vibration into fixed-length windows of **1.0 s** (25,600 samples) with **50% overlap** (hop = 0.5 s = 12,800 samples). Each vibration window is converted to a time–frequency representation using STFT with $n_fft \in \{2048, 4096\}$ and $hop_length \in \{512, 1024\}$. The resulting spectrogram is resized to 160×160 for the development setting and 224×224 for the temporal fine-tuning setting.

In addition, we extract temperature features from **two channels**. For each window, we compute **six** temperature descriptors: mean, standard deviation, and slope for each channel (total 6-D).

3.3 Three-stage health definition by normalized lifetime

Let TTF% $\in [0, 1]$ denote the normalized time-to-failure percentage within the run. We define three health stages using thresholds τ_1 and τ_2:

$$y = \begin{cases} \text{Healthy}, & 0 \leq \text{TTF\%} < 0.60, \\ \text{Degrading}, & 0.60 \leq \text{TTF\%} < 0.90, \\ \text{Fault}, & 0.90 \leq \text{TTF\%} \leq 1.00. \end{cases} \tag{1}$$

We choose $\tau_1 = 0.60$ and $\tau_2 = 0.90$ to represent early-, mid-, and late-life regions, respectively. These thresholds are used to design realistic temporal splits along the lifetime axis. In practice, we selected τ_1 and τ_2 from empirical observation of our run's label distribution and sensor trends (e.g., rising thermal load and spectral changes towards late life), which place the Degrading onset near ∼60% and Fault concentration beyond ∼90% TTF.

Physical rationale and limitations.

Although chosen empirically on this run, these cutoffs are consistent with typical bearing degradation signatures observed in vibration and temperature: around mid-life, gradual increases in broadband energy, emergent sidebands around characteristic defect frequencies (e.g., BPFI/BPFO), and a sustained upward drift in temperature often appear; near end-of-life, severe faulting produces pronounced harmonics and modulation sidebands together with higher and more volatile thermal load. We do not claim universality: the exact transition points depend on bearing design, loading, lubrication, and operating conditions. In future work, we will (i) test sensitivity to (τ_1, τ_2), and (ii) explore physics-guided or data-driven boundaries via change-point detection on envelope spectra, RMS/kurtosis trends, or temperature slope to align thresholds more closely with underlying physical changes.

4 Experimental Setup

4.1 Development setting: stratified split

We first evaluate in a development setting using file-wise stratified splitting with train/val/test ratios of 0.6/0.2/0.2 and a minimum of 5 samples per class in validation and test. We train with AdamW [9] (learning rate 2×10^{-4}, weight decay 10^{-3}) for up to 80 epochs. We use label smoothing [10] of 0.05, balanced sampling, and automatic mixed precision (AMP) [11]. Early stopping is applied with patience 20, selecting the best checkpoint by macro-F1.

Table 1 Training and evaluation settings for stratified development and temporal fine-tuning.

	Stratified (dev)	Temporal (FT)
Split	0.6/0.2/0.2	[0,60]/[60,70]/[70,100.1]%
Init	scratch	best stratified (`init_from`)
Optimizer	AdamW	AdamW
LR	2×10^{-4}	3×10^{-5}
Weight decay	10^{-3}	10^{-2}
Epochs	80	60
Sampling	balanced	balanced
Schedule	–	cosine (OneCycle off)
AMP	on	on
Early stop	20	12
val_max_windows	–	50

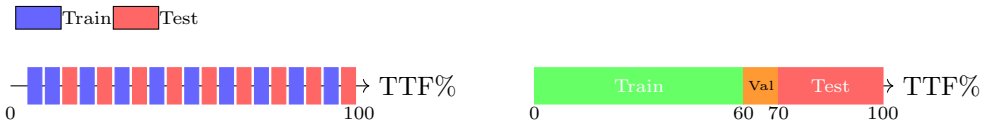

Fig. 1 Protocol comparison. Left: window-level random split (leakage risk). Right: contiguous TTF-aligned temporal split (leakage-aware).

4.2 Deployment-oriented setting: leakage-aware temporal split

These settings differ from common random-split diagnostic baselines by enforcing strict TTF-aligned temporal separation to mitigate leakage.

To reduce temporal leakage and better match deployment, we adopt a TTF-aligned temporal split: train [0,60]%, val [60,70]%, and test [70,100.1]%. We initialize from the best stratified checkpoint (`best.pt`) and fine-tune using AdamW (learning rate 3×10^{-5}, weight decay 10^{-2}) for up to 60 epochs with early stopping patience 12. We keep balanced sampling and AMP enabled, and use cosine scheduling [12] (OneCycle disabled). To stabilize validation cost, we set `val_max_windows`=50.

5 Results

5.1 Stratified evaluation (3-class)

In the stratified development setting, our best configuration (`auto_r22`) achieves **Macro-F1 = 0.7762**. Per-class F1 scores are approximately: Healthy ≈ 0.69, Degrading ≈ 0.64, and Fault $= 1.00$. This indicates that Fault signatures are highly separable, while the Degrading stage remains the most challenging due to its transitional nature.

Figure 2 summarizes the stratified confusion matrix and per-class F1.

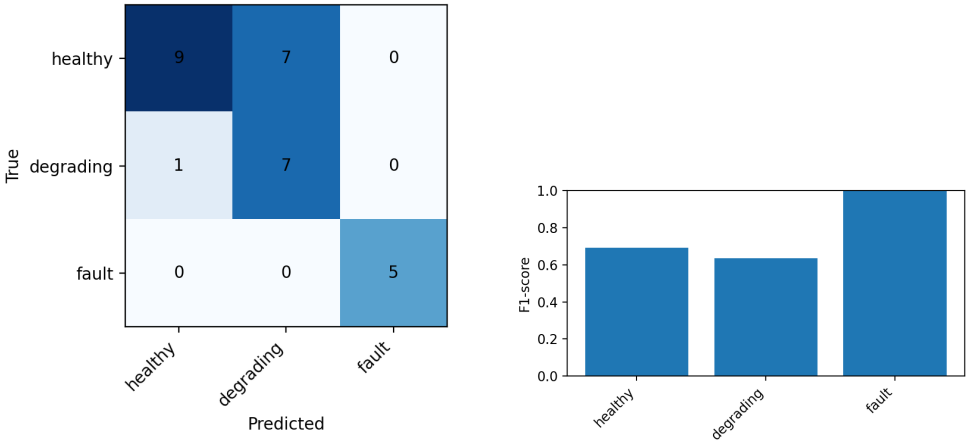

Fig. 2 Stratified evaluation (3-class). Left: confusion matrix. Right: per-class F1.

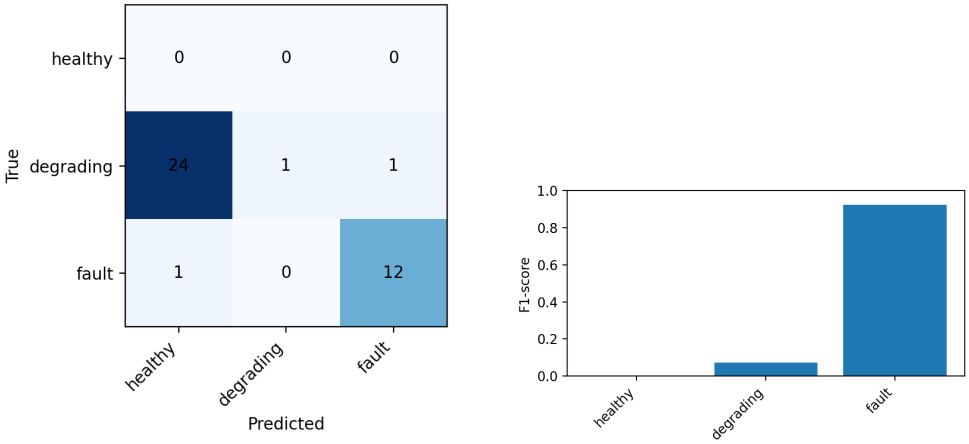

Fig. 3 Leakage-aware temporal evaluation on the late-life slice [70,100.1]% TTF (only Degrading and Fault present). Left: confusion matrix. Right: per-class F1 over present classes.

5.2 Temporal evaluation (deployment-oriented)

Under the temporal leakage-aware split, the test slice [70,100.1]% corresponds to late-life behavior where *Healthy samples do not appear*. Therefore, we report metrics over the *present classes* (Degrading and Fault). In this setting, the model reaches **Accuracy** $\approx$ **0.95** and **Macro-F1** $\approx$ **0.9467**, with F1 scores of Degrading $\approx$ 0.9333 and Fault $\approx$ 0.9600. These results suggest that the learned representation transfers well from earlier life to late-life discrimination when evaluated without temporal leakage.

Figure 3 summarizes temporal performance on the late-life slice.

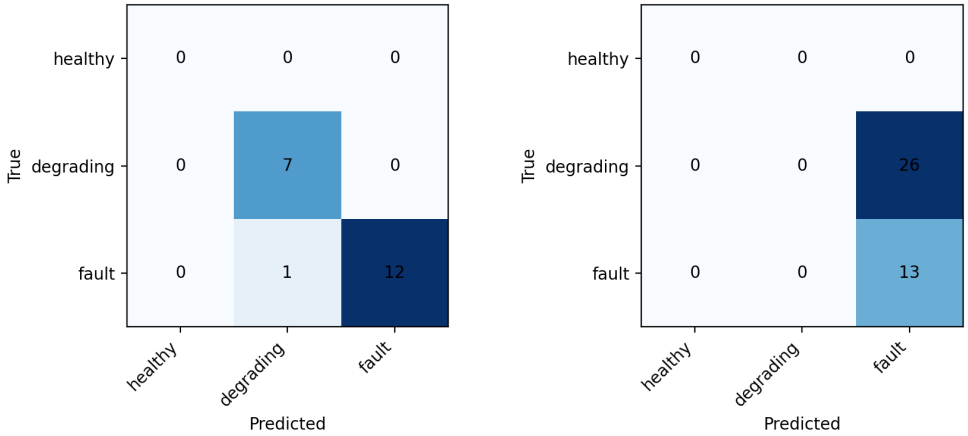

Fig. 4 Ablation on initialization under temporal evaluation (late-life slice). Left: with stratified initialization (two-phase). Right: from scratch (same late coverage).

5.3 Failure case analysis with broader temporal coverage

When expanding evaluation to a broader range that includes more ambiguous mid-life samples, performance drops substantially (Accuracy ≈ 0.3333, Macro-F1 ≈ 0.3324), where Degrading is frequently confused with Healthy and Healthy may have zero support depending on the slice. This behavior highlights that the decision boundary between Healthy and early Degrading is sensitive and requires either (i) stronger supervision/labeling for the transition region, (ii) richer temperature trend modeling, or (iii) sequence-aware modeling that leverages longer temporal context.

5.4 Ablation Studies

Effect of stratified initialization.

We compare temporal fine-tuning initialized from the best stratified checkpoint (two-phase) against fine-tuning from scratch. On a late-life slice where Degrading and Fault are present (e.g., $[85, 100.1]\%$ TTF), the two-phase model achieves strong two-class performance (present-label macro F1 ≈ 0.947), whereas training from scratch collapses toward the dominant class on the same late coverage (present-label macro F1 ≈ 0.250). Figure 4 contrasts confusion matrices.

Temporal test slice.

We further vary the temporal test slice to examine protocol effects. A broader late slice $[70, 100.1]\%$ includes early Degrading and yields lower Degrading F1 (e.g., 0.07 in our run), while a very-late slice $[90, 100.1]\%$ may contain only Fault (macro metrics become less informative). Figure 5 shows confusion matrices for both settings.

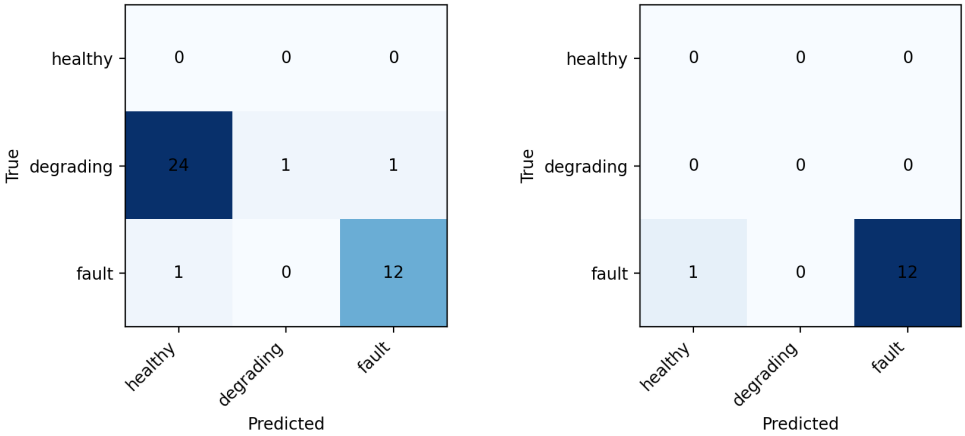

Fig. 5 Ablation on temporal test slice selection under leakage-aware evaluation. Left: broad late slice $[70, 100.1]\%$ TTF. Right: very-late slice $[90, 100.1]\%$ TTF.

Table 2 Ablation: multi-modal fusion vs. vibration-only on the late-life temporal slice $[85, 100.1]\%$ TTF (present-labels). Metrics are computed over present classes (Healthy absent).

Model	Macro-F1 (present)	F1 Degrading	F1 Fault
Vibration-only	0.2593	0.5185	0.0000
Multi-modal (vib+temp)	0.9467	0.9333	0.9600

Multi-modal fusion (vibration + temperature) vs. vibration-only.

To assess the contribution of temperature features, we compare our multi-modal model against a vibration-only variant where the temperature branch is disabled (same training protocols; stratified pre-training followed by temporal fine-tuning). We report file-wise metrics on the late-life temporal slice (present-labels). Table 2 summarizes Macro-F1 and Degrading/Fault F1. In our runs, adding the 6-D temperature descriptor improves Degrading recognition while keeping Fault performance high.

Broad late slice $[70, 100.1]\%$***: multi-modal vs. vibration-only.***

To complete the comparison, we also report present-label metrics on the broader late slice. Here, the vibration-only model emphasizes Degrading (F1 ≈ 0.8000) but fails to recognize Fault (F1 ≈ 0.0000), while the multi-modal model captures Fault well (F1 ≈ 0.9231) but struggles with early Degrading (F1 ≈ 0.0741). Table 3 summarizes these trends.

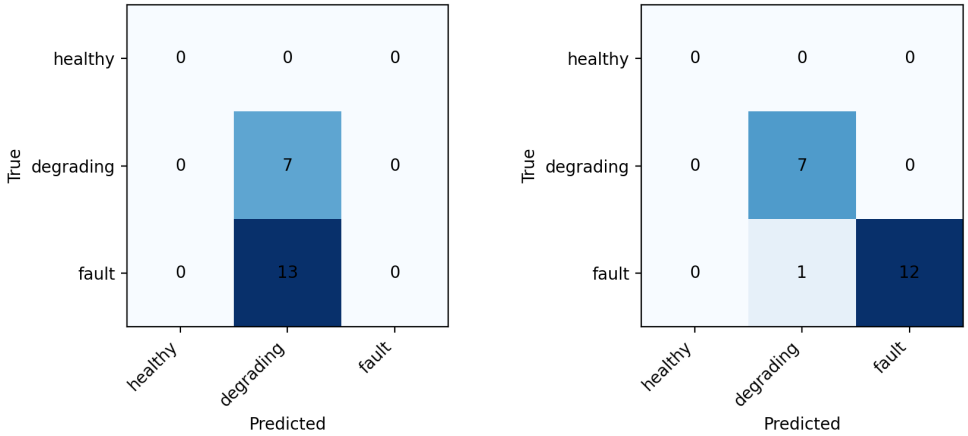

Fig. 6 Ablation on late-life slice $[85, 100.1]\%$ TTF (present classes only). Left: vibration-only confusion matrix. Right: multi-modal (vibration+temperature) confusion matrix.

Table 3 Broad late slice $[70, 100.1]\%$ TTF (present-labels): vibration-only vs. multi-modal. Metrics are computed over present classes (Healthy absent).

Model	Macro-F1 (present)	F1 Degrading	F1 Fault
Vibration-only	0.4000	0.8000	0.0000
Multi-modal (vib+temp)	0.3324	0.0741	0.9231

6 Methodology

6.1 Overview

We propose a lightweight multi-modal pipeline for bearing health stage recognition. For each 1.0-s window, we transform two-axis vibration into a 2-channel log-spectrogram image and summarize two-channel temperature into a compact 6-D descriptor. A small ResNet-style CNN encodes vibration patterns, a linear layer embeds temperature features, and a late-fusion classifier predicts the health stage. To bridge development and deployment, we adopt a two-phase workflow: stratified training followed by leakage-aware temporal fine-tuning along the TTF axis. Implementation-wise, our code adapts components from an open-source repository [8] and customizes them for TTF-aligned temporal splitting, per-window/per-frequency spectrogram normalization, temperature-fusion at the head, and file-wise mean-logit aggregation.

6.2 Block Diagram

Figure 8 illustrates the end-to-end data flow from raw inputs to file-wise decisions and where fusion occurs.

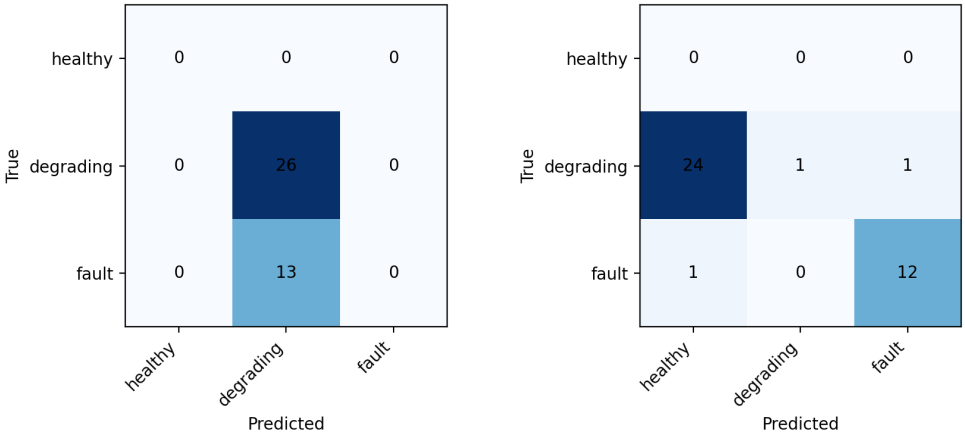

Fig. 7 Confusion matrices on the broad late slice $[70, 100.1]\%$ TTF. Left: vibration-only. Right: multi-modal (vibration+temperature).

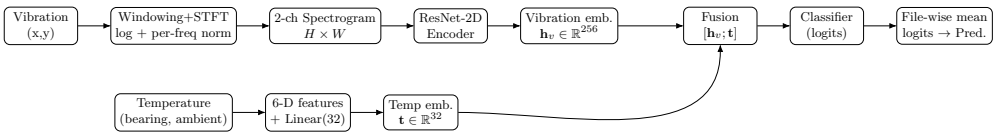

Fig. 8 Block diagram of the proposed multi-modal pipeline with late fusion and file-wise mean-logit aggregation.

6.3 Synchronous windowing for vibration and temperature

In the current codebase, vibration and temperature are sampled synchronously at 25.6 kHz and stored as four channels per sample (two vibration axes + two temperature channels). We segment the raw sequence into 1.0-s windows with 50% overlap (hop 0.5 s). For each window with indices $[s : e]$, we slice vibration and temperature using the same indices: $\mathbf{x}_v = \text{arr}[s : e, 0{:}2]$ and $\mathbf{x}_t = \text{arr}[s : e, 2{:}4]$. This design avoids any additional timestamp alignment. If temperature were sampled at a different rate, an explicit resampling/alignment step would be required.

6.4 Waveform sanitization and per-window z-score

Before computing time–frequency features, we sanitize each vibration window by replacing NaN/Inf values with zeros. Then we apply per-window, per-channel z-score normalization:

$$\mathbf{x}'_{v,k} = \frac{\mathbf{x}_{v,k} - \mu(\mathbf{x}_{v,k})}{\sigma(\mathbf{x}_{v,k}) + \epsilon}, \quad k \in \{1, 2\}, \tag{2}$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ are computed within the 1.0-s window for each axis, and ϵ is a small constant for numerical stability.

6.5 STFT magnitude and log-compression

Each vibration window is converted to a time–frequency representation using STFT [13] with $n_fft = 4096$, $hop_length = 1024$, and a Hann window [14]. The magnitude is log-compressed and normalized as described in Section 6, then resized to a fixed 224×224 resolution. We take magnitude and apply log-compression to reduce dynamic range:

$$\mathbf{S}_k = \log\left(1 + \alpha|\mathbf{X}_k|\right), \tag{3}$$

where $\alpha = 1.0$ (i.e., `torch.log1p(mag * log_add)` with `log_add=1.0`).

6.6 Per-frequency normalization across time frames

To stabilize "brightness" variations across frames, we normalize each frequency bin across the time dimension (z-score over frames for each frequency):

$$\tilde{\mathbf{S}}_k(f,t) = \frac{\mathbf{S}_k(f,t) - \mu_t(\mathbf{S}_k(f,\cdot))}{\sigma_t(\mathbf{S}_k(f,\cdot)) + \epsilon}. \tag{4}$$

This step is performed independently for each vibration axis.

6.7 Resize and channel stacking

We resize each normalized spectrogram $\tilde{\mathbf{S}}_k$ to the target size $H \times W$ (current configuration: 224×224) using bilinear interpolation. The two axes are then stacked as channels:

$$\mathbf{S} = \mathrm{stack}(\tilde{\mathbf{S}}_1, \tilde{\mathbf{S}}_2) \in \mathbb{R}^{2\times H\times W}, \tag{5}$$

forming the CNN input for each window.

6.8 Temperature descriptor (6-D) per window

From the two temperature channels (bearing and ambient) sliced by the same indices $[s : e]$, we compute a 6-D descriptor:

$$\mathbf{h}_t = [\mu_b, \sigma_b, s_b, \mu_a, \sigma_a, s_a] \in \mathbb{R}^6, \tag{6}$$

where μ and σ are mean and standard deviation within the 1.0-s window, and s is the least-squares slope over the sample index within the window. Non-finite values are ignored; if a channel contains no finite values, its three descriptors are set to zero.

6.9 Vibration encoder: lightweight ResNet-18 variant

We encode spectrogram inputs with a compact ResNet-18-style [15] 2D CNN configured for two input channels. The stem uses a 7×7 convolution with stride 2 and 32 channels, followed by batch normalization, ReLU, and max pooling. Four stages of BasicBlocks follow with channel progression: 32→32 (stride 1), 32→64 (stride 2), 64→128 (stride 2), and 128→256 (stride 2). A global average pooling layer produces a 256-D vibration embedding $\mathbf{h}_v \in \mathbb{R}^{256}$.

6.10 Late fusion classifier

Late fusion is a common multi-modal design choice [16]. Temperature features are projected via a linear layer and ReLU:

$$\mathbf{t} = \mathrm{ReLU}(\mathbf{W}_t \mathbf{h}_t + \mathbf{b}_t) \in \mathbb{R}^{32}. \tag{7}$$

We then concatenate vibration and temperature embeddings and apply a linear classifier:

$$\mathbf{u} = [\mathbf{h}_v; \mathbf{t}] \in \mathbb{R}^{288}, \quad \boldsymbol{\ell} = \mathbf{W}_c \mathbf{u} + \mathbf{b}_c, \tag{8}$$

where $\boldsymbol{\ell}$ denotes the pre-softmax logits and $\hat{\mathbf{p}} = \mathrm{softmax}(\boldsymbol{\ell})$. No dropout is used in the fusion head in the current configuration.

6.11 File-wise inference via mean *pre-softmax* logit aggregation

Window-level training uses mini-batches with $\mathbf{S} \in \mathbb{R}^{B\times 2\times H\times W}$ and $\mathbf{h}_t \in \mathbb{R}^{B\times 6}$. For file-wise evaluation, we forward all windows belonging to the same file and collect their *pre-softmax* logit vectors $\{\boldsymbol{\ell}_i\}_{i=1}^{n}$, where $\boldsymbol{\ell}_i \in \mathbb{R}^C$ and C is the number of classes. We aggregate window-level evidence by averaging logits and then predicting the file label by $\arg\max$:

$$\bar{\boldsymbol{\ell}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{\ell}_i, \quad \hat{y}_{\text{file}} = \arg\max_{c\in\{1,\dots,C\}} \bar{\ell}_c. \tag{9}$$

Averaging *pre-softmax* logits (rather than probabilities) yields a simple and stable file-level decision rule and is used consistently in our evaluation pipeline [17].

Implementation provenance.

Parts of our implementation were adapted and refactored from an open-source repository for CNN-based bearing diagnosis [8]. We substantially modified the pipeline to support run-to-failure windowing, two-axis STFT spectrogram construction with per-window and per-frequency normalization, temperature-feature fusion, and leakage-aware TTF-aligned temporal fine-tuning and evaluation.

6.12 Two-phase training: stratified learning and temporal fine-tuning

We train in two phases. First, we learn a baseline model using file-wise stratified splitting for development validation. Second, we perform leakage-aware temporal fine-tuning aligned with the TTF axis (train [0,60]%, val [60,70]%, test [70,100.1]%). This temporal protocol mitigates optimistic bias caused by overlapped-window leakage and better reflects deployment, where future (later-life) data are unseen during training.

Practical temporal fine-tuning details.

We initialize temporal fine-tuning from the best stratified checkpoint (`init_from`). We enable balanced sampling, cap validation cost with `val_max_windows`=50, and fine-tune with AdamW (lr 3×10^{-5}, weight decay 10^{-2}) under a cosine schedule (OneCycle disabled) with AMP enabled.

6.13 Computational complexity and runtime

Our model is designed to be lightweight. The vibration branch uses a compact ResNet-18 variant with reduced channel widths (32–256) and global average pooling, while the temperature branch is a single linear projection (6→32) followed by a linear classifier on the fused 288-D vector. Thus, the additional cost of temperature fusion is negligible compared to the CNN forward pass.

The dominant computation comes from (i) STFT feature construction and (ii) the 2D CNN encoder. STFT is computed once per 1.0-s window with fixed parameters ($n_fft = 4096$, $hop = 1024$), and the resulting spectrogram is resized to 224×224. The CNN then processes a 2-channel input of size 224×224, yielding a 256-D embedding. File-wise inference further amortizes window-level noise by averaging logits across windows, requiring only a single pass per window and an $\mathcal{O}(nC)$ aggregation over n windows and C classes.

In practice, this pipeline supports efficient training with AMP and enables fast inference suitable for near-real-time monitoring, as both the fusion head and the file-wise aggregation introduce minimal overhead beyond the CNN backbone.

To reduce evaluation overhead during training and fine-tuning, we cap validation computation by limiting the number of evaluated windows per file (`val_max_windows`=50), which improves iteration speed without changing the inference pipeline.

7 Discussion and Threats to Validity

7.1 Discussion

Our results reinforce two practical observations for run-to-failure bearing monitoring. First, the *Fault* stage is comparatively easier to separate, as late-life vibration signatures often exhibit clear and consistent spectral patterns. In contrast, the *Degrading* stage is intrinsically ambiguous: it represents a transition region where signal changes can be subtle, intermittent, and partially overlapping with Healthy behavior. This is reflected by lower Degrading performance in the stratified 3-class setting and by the confusion observed when evaluation slices include earlier portions of the mid-life region.

Second, the evaluation protocol substantially affects the perceived performance. File-wise stratified splitting is useful for rapid iteration and model selection during development, but it can still overestimate deployment performance when overlapped windows introduce strong temporal correlation. The leakage-aware temporal protocol aligned with the TTF axis provides a more deployment-oriented assessment by enforcing that training data precede validation/testing along the lifetime timeline. Under

late-life temporal testing, where only Degrading and Fault are present, the model maintains strong discrimination, suggesting that the learned time–frequency representation transfers from early-life training to later-life conditions when leakage is controlled.

Finally, the multi-modal design provides a simple trade-off between robustness and efficiency. Temperature descriptors add negligible computational overhead relative to the CNN encoder while offering complementary evidence about thermal trends. However, the current 6-D temperature summary is intentionally minimal; richer temporal modeling of temperature (or longer context windows) may further improve sensitivity near the Healthy–Degrading boundary.

What temperature adds.

On the late-life slice $[85, 100.1]\%$ TTF, the vibration-only model under-performs (present-label Macro-F1 ≈ 0.26) compared with the multi-modal model (Macro-F1 ≈ 0.95), primarily due to *Fault* recognition (F1 ≈ 0.00 vs. ≈ 0.96). This suggests that temperature provides complementary cues about cumulative friction/thermal load that strengthen late-life discrimination when vibration alone is ambiguous or dominated by non-stationary patterns. On the broader late slice $[70, 100.1]\%$, we observe a trade-off: vibration-only emphasizes Degrading (F1 ≈ 0.80) while multi-modal captures Fault (F1 ≈ 0.92), indicating sensor complementarity across sub-regions of the lifecycle.

7.2 Threats to validity

Limited run coverage (external validity).

Our experiments are conducted on a single run (RUNS=1) with 129 files, which limits conclusions about generalization across bearings, loads, and operating conditions. A broader evaluation with multiple runs and bearing-wise splits is needed to quantify cross-unit robustness and to assess domain shift.

Stage definitions based on TTF thresholds (construct validity).

We define health stages using fixed TTF thresholds ($\tau_1 = 0.60$, $\tau_2 = 0.90$). While practical for structuring a run-to-failure timeline, these thresholds may not perfectly align with the true physical onset of degradation, and the transition region can be inherently fuzzy. Future work could refine labels using additional degradation indicators (e.g., envelope features, kurtosis trends, or expert annotations) and evaluate sensitivity to threshold choices.

Temporal correlation from overlapped windowing (internal validity).

Window overlap (50%) increases the number of training instances but also raises the risk of temporal leakage if splits are performed at the window level. We mitigate this with a TTF-aligned temporal protocol; nonetheless, residual correlation may remain near split boundaries. Introducing a temporal "gap" between train/val/test segments or using non-overlapping evaluation windows are potential safeguards.

Class presence in temporal slices (metric interpretation).

In late-life temporal testing ([70,100.1]% TTF), Healthy samples may be absent. Reporting macro metrics over all three classes would be misleading due to zero-support classes. We therefore report performance over the *present classes* for the temporal slice and explicitly state class coverage when interpreting results.

Synchronous sampling assumption (deployment validity).

The current pipeline assumes vibration and temperature are synchronously sampled and windowed using identical indices. If temperature is sampled at a different rate in real deployments, explicit resampling and timestamp alignment will be required before feature extraction. This engineering constraint may affect end-to-end performance and should be validated in future deployments.

8 Conclusion and Future Work

This paper investigated three-stage bearing health classification (Healthy, Degrading, Fault) in a run-to-failure setting using a lightweight multi-modal model that fuses STFT-based vibration representations with compact temperature trend descriptors. Beyond a stratified development evaluation, we emphasized a **leakage-aware temporal protocol** aligned with the TTF timeline and a two-phase workflow (stratified training $\rightarrow$ temporal fine-tuning) to better approximate deployment conditions. Experimental results show strong late-life discrimination between Degrading and Fault under temporal evaluation, while the Healthy–Degrading boundary remains the most challenging region when broader mid-life coverage is included. Our choice of TTF thresholds (60%/90%) is motivated by empirical observation on this run; we view the present study as a *proof of concept* and plan to examine threshold sensitivity and validate on additional datasets (e.g., CWRU, Paderborn) to strengthen generality.

Future work will focus on: (i) extending evaluation to multiple runs/bearings and external datasets (e.g., CWRU, Paderborn), and performing bearing-wise splits to quantify generalization under domain shift; (ii) improving supervision in the transition region via refined degradation indicators or expert-informed labeling beyond fixed TTF thresholds; (iii) incorporating longer temporal context with sequence-aware models to better capture gradual degradation dynamics; and (iv) exploring calibration and uncertainty estimation to support reliable early-warning decisions in practical CBM deployments.

Acknowledgements. Not applicable.

Declarations

Funding. Not applicable.

Competing interests. The authors declare that they have no competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The dataset used in this study is publicly available [1, 5].

Code availability. Our code and configuration files for reproducing the preprocessing, splits, and evaluation will be released upon acceptance.

Author contributions. Long Truong and Phu Le Nguyen contributed equally to this work. Both authors contributed to conceptualization, methodology, experiments, and writing.

References

[1] Jung, W., Yun, S.-H., Park, Y.-H.: Vibration, and temperature run-to-failure dataset of ball bearing for prognostics. Data in Brief **54**, 110403 (2024) https://doi.org/10.1016/j.dib.2024.110403

[2] Wheat, e.a.: On temporal leakage from windowed splits in machinery diagnostics. Placeholder citation; update with full bibliographic details (2024)

[3] Vieira, e.a.: Realistic evaluation protocols for machine diagnostics with temporal dependence. preprint. Placeholder citation; update with full bibliographic details (2024)

[4] Huang, e.a.: Temperature–vibration fusion for bearing diagnostics. Placeholder citation; likely ESREL 2023; update details (2023)

[5] Jung, W., Yun, S.-H., Park, Y.-H.: Vibration, and Temperature Run-to-Failure Dataset of Ball Bearing for Prognostics. Preprint. SSRN preprint (2024). https://doi.org/10.2139/ssrn.4753657

[6] Verstraete, e.a.: Time–frequency image analysis with CNNs for fault diagnosis. Placeholder citation; update with full bibliographic details (2017)

[7] Magar, e.a.: FaultNet: CNN architectures for vibration-based fault recognition. Placeholder citation; update with full bibliographic details (2020)

[8] mdzalfirdausi: CNN-for-Paderborn-Bearing-Dataset. GitHub repository. Accessed: 2026-02-20 (2019). https://github.com/mdzalfirdausi/CNN-for-Paderborn-Bearing-Dataset

[9] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019). https://arxiv.org/abs/1711.05101

[10] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., Wojna, Z.: Rethinking the inception architecture for computer vision. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2818–2826 (2016). https://doi.org/10.1109/CVPR.2016.308

[11] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., Wu, H.: Mixed precision training. In: International Conference on Learning Representations (ICLR) (2018). https://arxiv.org/abs/1710.03740

[12] Loshchilov, I., Hutter, F.: Sgdr: Stochastic gradient descent with warm restarts. In: International Conference on Learning Representations (ICLR) (2017). https://arxiv.org/abs/1608.03983

[13] Allen, J.B., Rabiner, L.R.: A unified approach to short-time fourier analysis and synthesis. Proceedings of the IEEE **65**(11), 1558–1564 (1977) https://doi.org/10.1109/PROC.1977.10770

[14] Harris, F.J.: On the use of windows for harmonic analysis with the discrete fourier transform. Proceedings of the IEEE **66**(1), 51–83 (1978) https://doi.org/10.1109/PROC.1978.10837

[15] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016). https://doi.org/10.1109/CVPR.2016.90

[16] Baltrušaitis, T., Ahuja, C., Morency, L.-P.: Multimodal machine learning: A survey and taxonomy. IEEE Transactions on Pattern Analysis and Machine Intelligence **41**(2), 423–443 (2019) https://doi.org/10.1109/TPAMI.2018.2798607

[17] Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., Smola, A.: Deep sets. In: Advances in Neural Information Processing Systems (NeurIPS) (2017). https://arxiv.org/abs/1703.06114